Tarea 1 - IIC2560

Objetivos

Parsear un string

Modificar nodos de la gramática

Encontrar todos los árboles posibles

Aprender a programar en distintos lenguajes

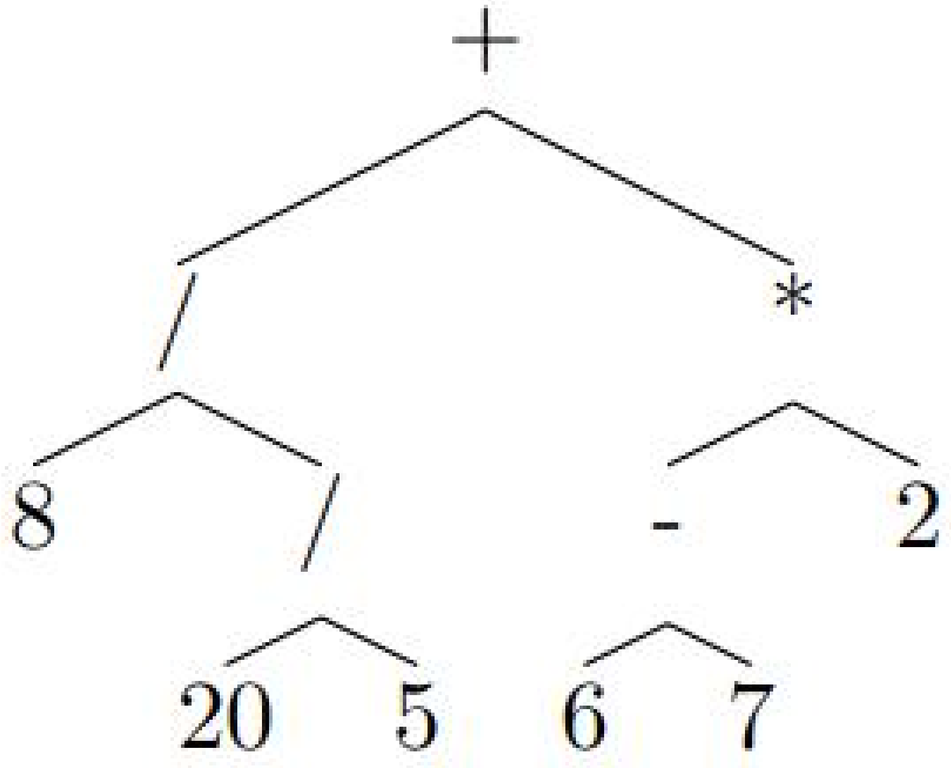
Intro a Parsing

Cómo parsearían un string de la siguiente gramática?

Considere la siguiente gramática:

$$
\begin{aligned}
\langle expression \rangle ::= \ & (\langle expression \rangle + \langle expression \rangle) \\
| \quad & (\langle expression \rangle \text{ - } \langle expression \rangle) \\
| \quad & (\langle expression \rangle * \langle expression \rangle) \\
| \quad & (\langle expression \rangle \ / \ \langle expression \rangle) \\
| \quad & \langle number \rangle
\end{aligned}
$$

```
((8 / (20 / 5)) + ((6 - 7) * 2))
```



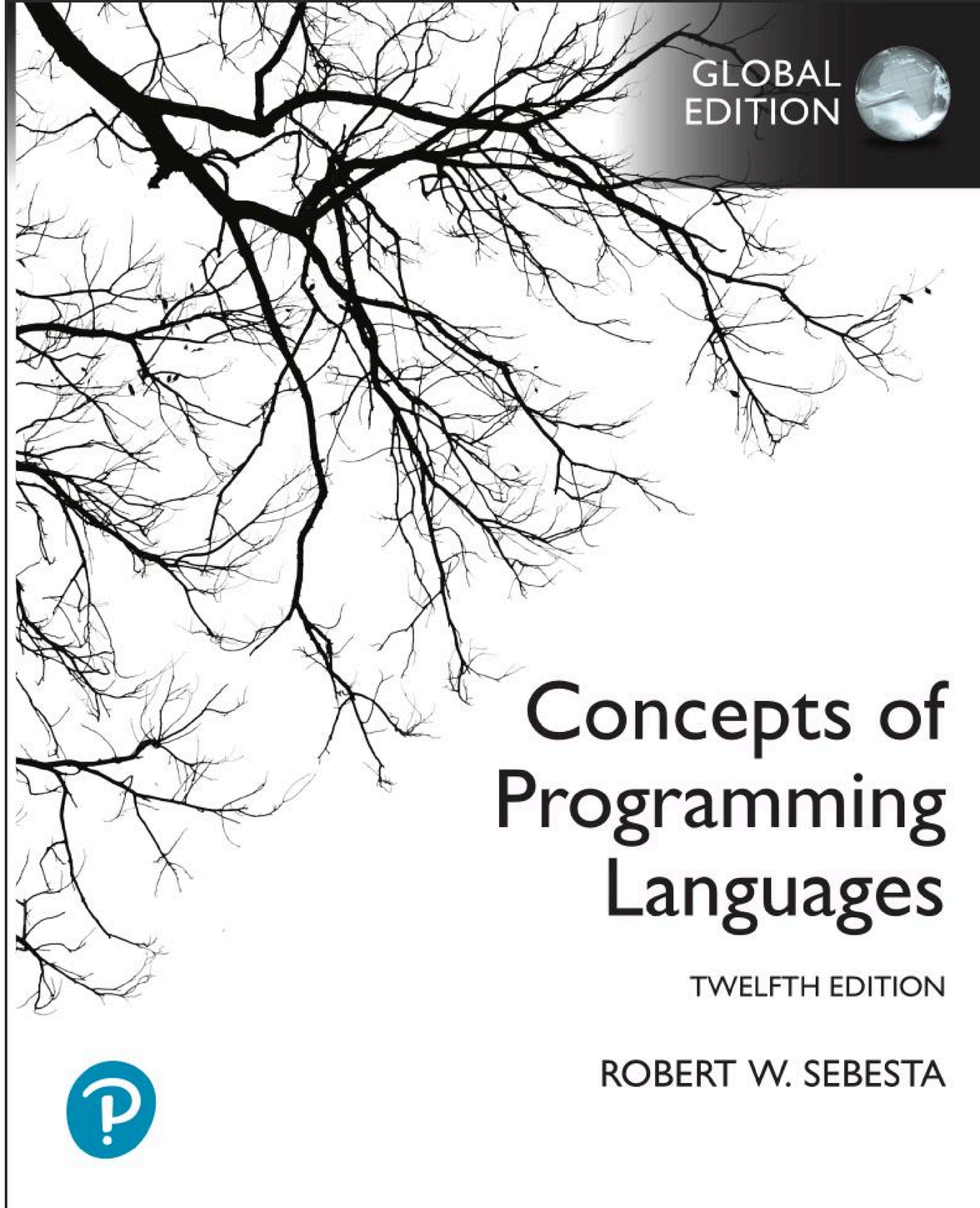
Considere la siguiente gramática:

$$
\begin{aligned}
\langle expression \rangle ::= \; & (\langle expression \rangle + \langle expression \rangle) \\
\mid \; & (\langle expression \rangle \text{ - } \langle expression \rangle) \\
\mid \; & (\langle expression \rangle * \langle expression \rangle) \\
\mid \; & (\langle expression \rangle \; / \; \langle expression \rangle) \\
\mid \; & \langle number \rangle
\end{aligned}
$$

```
((8 / (20 / 5)) + ((6 - 7) * 2))
```

Revisen algunos caps del libro!

Sintáxis → Capítulo 3 al 3.4
Parsing → Capítulo 4.3 al 4.5
Bindings y Scope → Capítulo 5.4 al 5.6



Intro a Java

The Java programming language

- **High level**
- **Object Oriented**
- **General Purpose**
- **Memory Safe**
- **C like syntax**
- **Strongly typed**
- **Bytecode based**
- **Released 1995**

- En este curso usaremos Java para la mitad de la tarea 1 junto con el maven builder.
- Les entregamos una interfaz para que sepan desde donde correremos los tests.
- La otra mitad es en Python.

```java
void main() {
    IO.println("Hello world!");
    var name = IO.readln("What is your name? ");
    IO.println("Hello " + name);
}
```

You can run it in the same way as previously.

```
> java MyFirstJavaApp.java
```

It should ask you for your name, and then print the following.

```
Hello world!
What is your name? Mary
Hello Mary
```

Congratulations, you are now ready for complex applications!

Learn Java

Running Your First Java Application

Getting Started with Java

Downloading and setting up the JDK, writing your first Java class, and creating your first Java application.

Launching Simple Source-Code Programs

Launching simple source-code Java programs with the Java launcher.

JShell - The Java Shell Tool

jshell interactively evaluates declarations, statements, and expressions of the Java programming language in a read-eval-print loop (REPL).

Building a Java application in Visual Studio Code

Oracle Java Platform extension enables you to develop your Maven and Gradle Java project in Visual Studio Code.

Building a Java application in IntelliJ IDEA

Learn how to code, run, test, debug and document a Java application in IntelliJ IDEA.

Building a Java Application in the Eclipse IDE

Installing and getting started with the Eclipse IDE for developing Java applications

Tiene muy buena integración para IDEs.

Tienen plan de estudiantes en IntelliJ

The following program, `IfElseDemo`, assigns a grade based on the value of a test score: an A for a score of 90% or above, a B for a score of 80% or above, and so on.

```
class IfElseDemo {
    public static void main(String[] args) {

        int testscore = 76;
        char grade;

        if (testscore >= 90) {
            grade = 'A';
        } else if (testscore >= 80) {
            grade = 'B';
        } else if (testscore >= 70) {
            grade = 'C';
        } else if (testscore >= 60) {
            grade = 'D';
        } else {
            grade = 'F';
        }
        IO.println("Grade = " + grade);
    }
}
```

The output from the program is:

```
Grade = C
```

https://dev.java/learn/

Running Apache Maven

The syntax for running Maven is as follows:

```
mvn [options] [<goal(s)>] [<phase(s)>]
```

All available options are documented in the built-in help that you can access with

```
mvn -h
```

The typical invocation for building a Maven project uses a Maven lifecycle phase. E.g.

```
mvn verify
```

The built-in lifecycles and their most used phases, in order, are:

- clean - `clean`
- default - `validate`, `compile`, `test`, `package`, `verify`, `install`, `deploy`
- site - `site`, `site-deploy`

https://maven.apache.org/run.html

https://docs.oracle.com/en/java/javase/26/docs/api/index.html

self == other

```
@Override
    public boolean equals(Object obj) {
        if(obj instanceof EqualsDemoSample) {
            EqualsDemoSample equalsSample
                = (EqualsDemoSample) obj;
         if(equalsSample.getName().equals(this.getName())) {
                return true;
            }
        }
        return false;
```

Java

https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/lang/Object.html#equals(java.lang.Object)

```
def __eq__(self, other):
    """Overrides the default implementation"""
    if isinstance(other, Number):
        return self.number == other.number return False
```

Python

https://docs.python.org/3/reference/datamodel.html#object.__eq__

python > parser > parser.py > Parser > parse > program_string

You, now | 1 author (You)

```python
class Parser:

    @staticmethod
    def parse(program_string):      You, now • Uncommitted changes
        pass
```

java > parser > src > main > java > com > iic2560 > Parser.java > Parser > parse(String)

You, 37 minutes ago | 1 author (You)

```java
package com.iic2560;

import com.iic2560.Nodes.Ast;

```

You, 37 minutes ago | 1 author (You)

```java
public class Parser {
    public static Ast parse(String program){
        return null;
    }
}
```